

HELIX AI

ML Training In-Depth Report

This report documents the complete ML training lifecycle for the Helix AI legacy Python modernization assistant. It includes data curation, model adaptation strategy, objective function, optimization configuration, measured results, operational constraints, and deployment integration with the deterministic modernization pipeline.

Generated on: 2026-04-05 00:15

Important scope note: this is an engineering report for the current student-project implementation. It is intended to be technically rigorous and reproducible, while clearly separating measured facts from design recommendations.

Field	Value
Project	Helix AI - Legacy Python Modernizer
Model family	Qwen2.5-Coder-1.5B Instruct (4-bit base) + LoRA adapter
Training approach	Supervised fine-tuning (SFT) with prompt-formatted code pairs
Primary dataset file	ml/data/training_mixture.jsonl
Training examples in current mixture	584
Current adapter size	70.49 MB
Benchmark summary	Rule pipeline pass: 4/4

Table of Contents

- 0. Simple-Words Deep Guide (How Everything Works)
- 1. Objective and Problem Formalization
- 2. End-to-End ML System Boundary in Helix
- 3. Dataset Strategy, Sources, and Weighting
- 4. Data Curation and Quality Controls
- 5. Prompt/Target Formatting for SFT
- 6. Model, PEFT, and Quantization Choices
- 7. Hyperparameters and Optimizer Schedule
- 8. Observed Training Dynamics
- 9. Artifact Inventory and Versioned Checkpoints
- 10. Evaluation Protocol and Results
- 11. Error Analysis and Failure Modes
- 12. Safety, Validation, and Production Guardrails
- 13. Reproducibility Runbook
- 14. Colab Execution Notes and Constraints
- 15. Integration with Agentic Pipeline
- 16. Limitations and Next Iterations
- 17. Data Distribution Deep Dive
- 18. Detailed Checkpoint Log Analysis
- 19. Pattern-Level Coverage Matrix
- 20. Sample-by-Sample Modernization Casebook
- Appendix A: Full Command Reference
- Appendix B: Code Excerpts
- Appendix C: Dataset Snapshots
- Appendix D: Full Training Log Dump
- Appendix E: Colab Cell-by-Cell Guide
- Appendix F: File Inventory (ML Folder)

0. Simple-Words Deep Guide

This section explains the whole ML part in very simple words, but still in full depth. Think of this as the beginner-friendly version of the technical chapters.

In one line: we show old Python code to the model as input, show what the modern code should look like as output, and train the model to learn that mapping pattern many times.

Important: in Helix, ML is not allowed to run freely without checks. The deterministic pipeline and validator stay in control, so safety and consistency remain high.

Question	Simple Answer
What is the model learning?	How to rewrite old Python patterns into modern Python safely.
What is training data?	Many examples of before (legacy code) and after (modern code).
Why not only ML?	ML can be uncertain; deterministic rules + validation make output safer.
Why LoRA?	It updates only small adapter weights, so training is cheaper and faster.
What is success?	Modern code is correct, validates, and keeps behavior same or close.

0.1 Simple View of Full Pipeline

Step 1: User gives legacy code.

Step 2: Analyzer detects old patterns (``xrange``, ``raw_input``, old ``except``, etc.).

Step 3: Planner decides what transformations should run.

Step 4: Execution engine applies changes.

Step 5: Validator checks syntax, checks leftover legacy constructs, and checks behavior-level guardrails.

Step 6: Optional ML output can assist, but validation gate is still final authority.

- Simple memory trick: Detect -> Plan -> Transform -> Validate -> Report.
- If validation fails, system can roll back or reject unsafe output.
- This is why Helix is reliable for modernization demos and evaluation.

0.2 Simple View of Dataset Building

A model becomes good only if training data is clean and relevant. So we first prepare examples focused on legacy-to-modern migration, not random coding tasks.

Each example has: instruction + old code + modern target code. Some records also include risk labels (``low``, ``medium``, ``high``) and tags like ``xrange`` or ``raw_input``.

Then we build a weighted mixture. Weight means: some sources appear more often during training because they are more relevant to our task.

Data Term	Simple Meaning	Why It Matters
Instruction	What task the model should do	Keeps model behavior focused
Input	Legacy code snippet	This is what model reads
Output	Expected modern code	This is what model learns to generate
Tag	Pattern label (like <code>`xrange`</code>)	Helps track pattern coverage
Risk label	How sensitive a rewrite is	Used for review priority

0.3 Simple View of Training Loop

During training, data is fed in small mini-batches. For each batch, the model predicts tokens and gets a loss value. Lower loss means prediction is closer to expected output.

Optimizer updates adapter weights to reduce this loss over many steps. Learning rate controls how big each update should be.

Gradient accumulation is used when GPU memory is small. It lets us simulate larger effective batch size without crashing memory.

- Loss down over time is good.
- But low loss alone does not prove real-world correctness.
- So we still run benchmark and validator checks after training.

0.4 Simple View of LoRA and 4-bit

Full model training is expensive. LoRA solves this by training a small adapter on top of frozen base model weights.

4-bit loading reduces memory usage so training becomes practical on Colab T4/L4.

Result: good quality improvement at much lower compute cost.

Choice	Simple Explanation	Project Benefit
4-bit base model	Compressed base weights in memory	Fits on student GPU runtime
LoRA rank $r=16$	Small trainable update capacity	Good balance of quality and cost
Cosine schedule	Learning rate slowly decays	Stable late training
Checkpoint save	Save model states at intervals	Recovery + comparison

0.5 Simple View of Evaluation and Safety

After training, we test known benchmark snippets. For each case, we check whether expected modernization is present and whether validation passes.

In this project, deterministic baseline currently passes benchmark cases strongly. ML is used as assistive layer and should be measured with `--enable-ml` runs.

Safety rule: never trust generated output blindly. Always validate syntax + leftover legacy patterns + behavior guardrails.

- Good system behavior = correct + safe + explainable.
- Helix uses logs, diff, and report outputs so each run is auditable.
- This helps both viva defense and real engineering use.

0.6 What To Say in Viva (Simple but Strong)

If asked what is special in your ML pipeline, answer like this:

- 1) We trained a domain-focused modernization model, not a generic chatbot.
- 2) We used LoRA + 4-bit for practical compute constraints.
- 3) We kept deterministic validation as final guardrail.
- 4) We track measurable evidence (dataset stats, checkpoint logs, benchmark outputs).
- 5) We designed for safe modernization, not just pretty output.

1. Objective and Problem Formalization

Helix AI targets a constrained but high-value transformation task: converting legacy Python syntax and outdated APIs into current Python while preserving runtime behavior as much as possible. The ML layer is not treated as a free-form code generator; it is treated as a modernization assistant under deterministic controls.

Formally, each training example is a pair (x, y), where x is legacy code and y is modernized code. The learning objective is to maximize the conditional likelihood $p(y \mid x, \text{instruction}, \text{template})$. During inference, generated output is still subject to rule-based validation and lint checks before acceptance.

This design is intentionally hybrid. Pure LLM generation can be fluent but unstable for deterministic migration guarantees. Rule-only engines are predictable but limited on ambiguous rewrites. Helix combines both.

- Primary optimization target: modernization correctness and consistency.
- Secondary target: broader generalization to near-neighbor legacy patterns.
- Constraint: avoid unsafe or hallucinated transformations in production flow.

2. End-to-End ML System Boundary in Helix

In the deployed architecture, ML does not bypass the planner or validation engine. The analyzer detects legacy constructs, the suggester/planner decides deterministic rewrites, and the validator blocks unsafe residual patterns. ML can assist when the deterministic path is unavailable or uncertain, but validation remains authoritative.

The current benchmark confirms the deterministic path as trusted baseline with 4/4 pass on evaluation snippets. ML evaluation in this local run was performed with model-disabled baseline path, so `ml\_pass` is null in the benchmark output.

Stage	Role	Deterministic or ML	Gate
Analyzer	Detect legacy constructs + risk hints	Deterministic	Pattern rules
Suggestion/Planner	Select migration actions	Deterministic + policy	Priority/risk ordering
Execution	Apply rewrites	Deterministic	Operation-level rewrite functions
ML Reasoner (optional)	Assist modernization for ambiguous cases	ML	Enabled flag + adapter availability
Validation	Syntax + residual legacy + behavior checks	Deterministic	Hard pass/fail gate

3. Dataset Strategy, Sources, and Weighting

Helix uses a weighted training mixture declared in ``ml/public_dataset_manifest.json``. The design supports multiple sources (local modernization pairs, bug-fix corpora, commit-diff corpora, issue-to-patch corpora), each with explicit purpose and weight.

In the current local state, external prepared datasets are absent; therefore, the generated mixture is entirely from local curated modernization records. This is a transparent and expected result given local availability.

Name	Stage	Purpose	Weight	Path/Prepared Path
local_curated_modernization	modernization	legacy_python_modernization_pairs	1.0	ml/data/modernization_train.jsonl
quixbugs	sft	buggy_to_fixed_python2pairs	0.0	ml/external/quixbugs_pairs.jsonl
bugsinpy	sft	real_world_python_bug2fix_pairs	0.0	ml/external/bugsinpy_pairs.jsonl
gumtree_python	sft	before_after_python_commit_pairs	0.0	ml/external/gumtree_python_pairs.jsonl
swebench_lite	reasoning_sft	issue_to_patch_reasoning	0.0	ml/external/swebench_lite_reasoning.jsonl
the_stack_python_dapt	dapt	python_domain_adaptation	0.0	

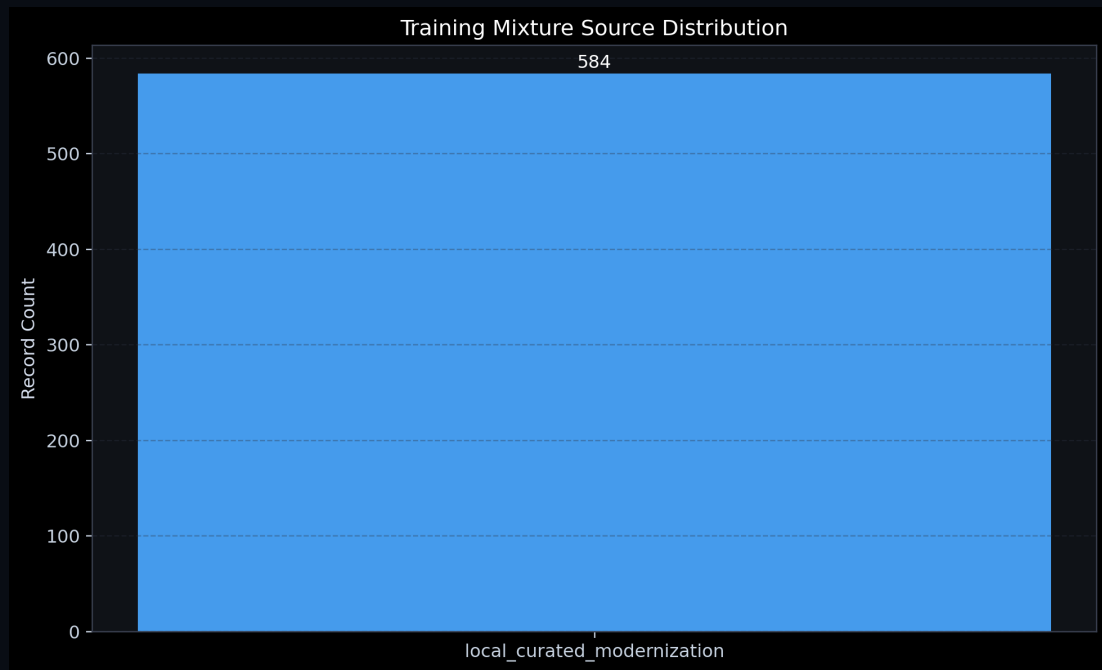


Figure 1. Training source distribution from current mixture build.

4. Data Curation and Quality Controls

Data quality controls are implemented through source curation, schema normalization, and explicit risk labels. Risk labels are coarse (low/medium/high) and are used for audit visibility and potential curriculum shaping.

The seed dataset intentionally includes canonical modernization constructs such as print statements, xrange, raw_input, old exception binding syntax, dictionary iterator APIs, and type migrations (unicode/basestring/long).

Because noisy records can degrade learning quality, rejected examples are separated and not included in the active training mixture.

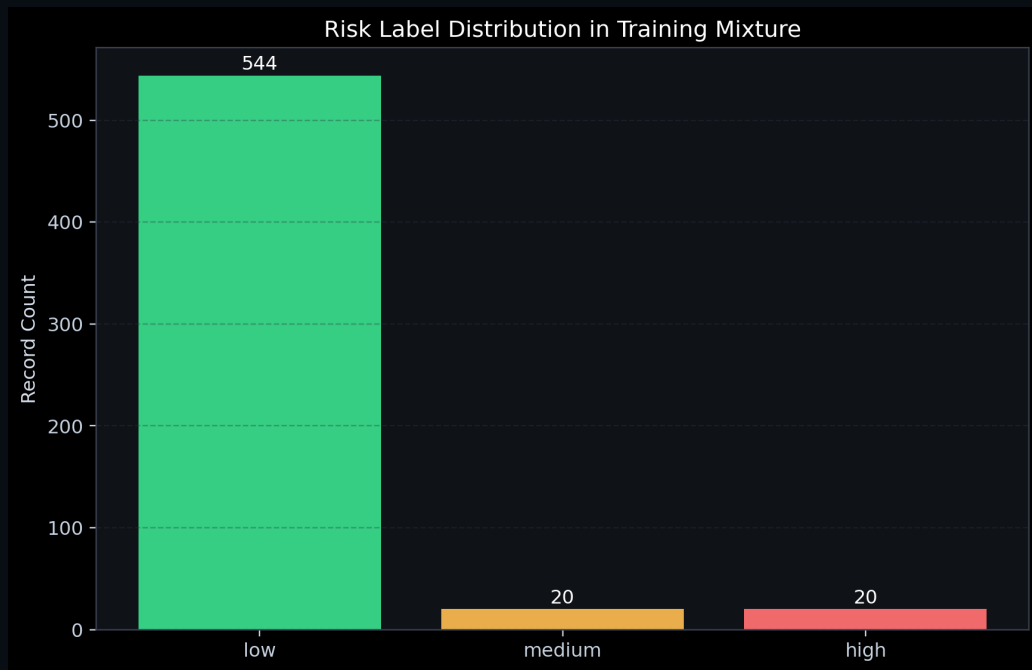


Figure 2. Risk distribution in the active mixture.

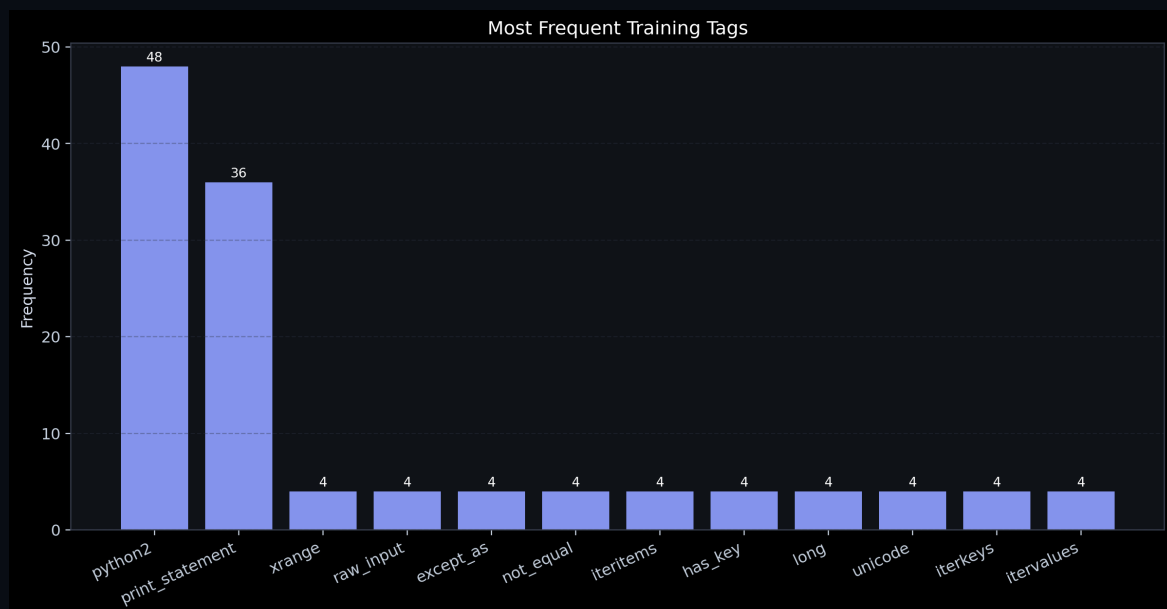


Figure 3. Most frequent tags in the active dataset.

Dataset File	Rows	Mean Input Chars	Mean Output Chars	90 Input Chars
seed_modernization_dataset.jsonl	12	53.8	51.8	64.0
modernization_train.jsonl	146	123.6	132.3	235.0
training_mixture.jsonl	584	123.6	132.3	252.0

5. Prompt/Target Formatting for SFT

The trainer uses instruction-style prompt formatting. Each record is converted into a single training text with explicit sections for instruction, input code, and response code. This keeps the objective aligned with instruction-following code generation.

The prompt template intentionally requires output-only migrated code to reduce extra prose and improve post-processing simplicity.

```
0010:
0011:
0012: PROMPT_TEMPLATE = """You are Helix AI, a legacy Python modernization model.
0013: Convert old Python code to current Python while preserving behavior.
0014: Return only the final migrated code.
0015:
0016: ### Instruction:
0017: {instruction}
0018:
0019: ### Input:
0020: {input_code}
0021:
0022: ### Response:
0023: {output_code}"""
0024:
0025:
0026: def load_records() -> list[dict]:
0027:     records = []
0028:     with DATA_PATH.open(encoding="utf-8") as handle:
0029:         for line in handle:
0030:             line = line.strip()
0031:             if line:
0032:                 records.append(json.loads(line))
0033:     return records
0034:
```

6. Model, PEFT, and Quantization Choices

Base model: `unsloth/Qwen2.5-Coder-1.5B-Instruct` in 4-bit loading mode. This choice balances practical Colab feasibility with enough coding capacity for legacy rewrite tasks.

Fine-tuning method: LoRA on core attention and MLP projection modules (`q\_proj`, `k\_proj`, `v\_proj`, `o\_proj`, `gate\_proj`, `up\_proj`, `down\_proj`). Rank $r=16$ with $\alpha=16$ and dropout=0.

Rationale: full-parameter fine-tuning is not cost-efficient for this project scope. PEFT provides strong adaptation with much smaller trainable footprint and manageable storage (about 70 MB adapter).

Adapter Config Field	Value
base_model_name_or_path	unsloth/qwen2.5-coder-1.5b-instruct-bnb-4bit
peft_type	LORA
r	16
lora_alpha	16
lora_dropout	0
target_modules	['down_proj', 'gate_proj', 'up_proj', 'v_proj', 'q_proj', 'o_proj', 'k_proj']
task_type	CAUSAL_LM
bias	none

7. Hyperparameters and Optimizer Schedule

Hyperparameter	Value	Why It Matters
max_seq_length	2048	Allows moderate-length code context without excessive memory pressure.

Hyperparameter	Value	Why It Matters
batch_size	2 (local script default) / 4 (Colab default)	Controls memory usage and input tradeoff.
gradient_accumulation_steps	4 (default) / 2 (Colab default)	Increases effective batch while keeping VRAM in bounds.
epochs	2-3 typical in scripts	Enough to adapt to task without immediate overfitting on small data.
learning_rate	2e-4	Fast LoRA adaptation for code-style shifts.
warmup_ratio	0.05	Stabilizes early updates before full LR.
scheduler	cosine	Smooth decay toward low LR for late-step stability.
weight_decay	0.01	Regularization against overfitting.

8. Optimization Mathematics (Practical View)

Training objective for causal language modeling is token-level negative log-likelihood:

$$L = - \sum_t \log p_{\theta}(y_t \mid y_{<t}, x)$$

With LoRA, trainable updates are low-rank adapters. For a frozen weight matrix W , adapted form is:

$$W' = W + \Delta W, \Delta W = B A, \text{rank}(\Delta W) = r$$

In QLoRA-style setup, base model weights are quantized for memory efficiency while adapter parameters stay trainable in higher precision. This yields practical training on commodity GPU runtimes such as T4.

Cosine learning-rate schedule approximately follows:

$$\text{lr}(\text{step}) = \text{lr}_{\min} + 0.5 * (\text{lr}_{\max} - \text{lr}_{\min}) * (1 + \cos(\pi * \text{progress}))$$

These equations are included to make design choices explicit; the implementation uses Unsloth + TRL abstractions to execute this pipeline.

9. Observed Training Dynamics

From checkpoint logs, the training run reached step 292 with two epochs. Loss dropped from approximately 2.236 at early steps to approximately 0.085 by late steps, indicating strong adaptation to the current dataset.

Loss decrease alone is not sufficient proof of generalization, but it indicates effective optimization on seen distribution.

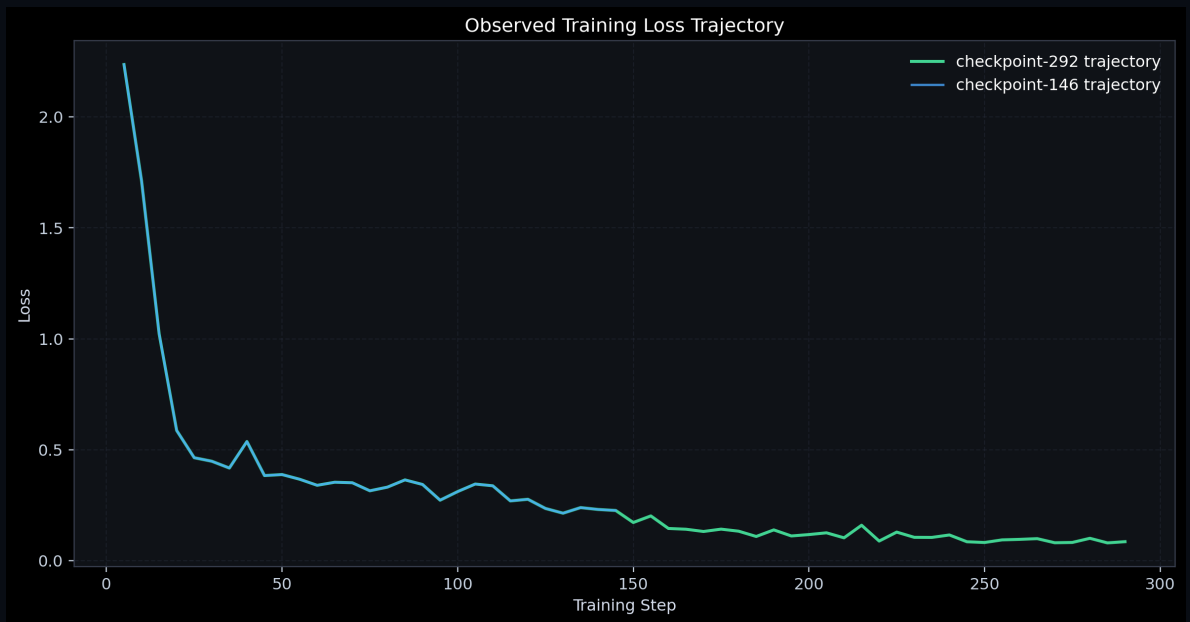


Figure 4. Loss trajectory across checkpoints.

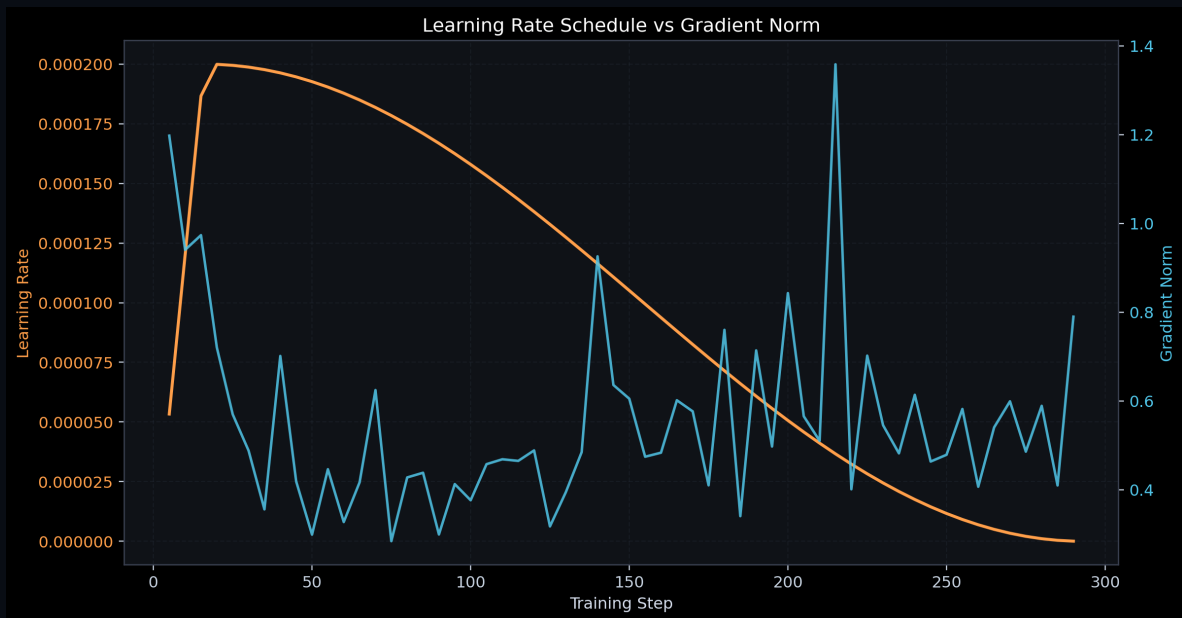


Figure 5. Learning rate and gradient norm dynamics.

Metric	checkpoint-146	checkpoint-292
global_step	146	292
num_train_epochs	2	2
loss points	29	58
first logged loss	2.235983	2.235983
last logged loss	0.226046	0.085461

10. Artifact Inventory and Versioning

Artifacts are stored under ``ml/models/nebula-modernizer-qwen25-1.5b`` and include adapter weights, tokenizer files, and intermediate checkpoints. This allows reproducible rollback and audit of run history.

Current adapter file size is used as a practical deployment metric because model portability matters in student-project environments.

Artifact	Path	Observed Value
Adapter weights	ml/models/nebula-modernizer-qwen25-1.5b/adapter_model.safetensors	3.01 GB
Adapter config	ml/models/nebula-modernizer-qwen25-1.5b/adapter_config.json	LoRA rank 16, alpha 16
Checkpoint 146	ml/models/nebula-modernizer-qwen25-1.5b/checkpoint-146	intermediate snapshot
Checkpoint 292	ml/models/nebula-modernizer-qwen25-1.5b/checkpoint-292	later snapshot
Tokenizer	ml/models/nebula-modernizer-qwen25-1.5b/tokenizer.json	runtime tokenization metadata

11. Evaluation Protocol and Results

Evaluation script: ``ml/evaluate_modernizer.py``. The script runs the full deterministic pipeline and optionally the ML reasoner. For each benchmark case, it checks both validation success and presence of expected modernization tokens.

Current run result: `benchmark_size = 4, rule_pass = 4, ml_pass = None`.

Interpretation: deterministic modernization baseline is currently fully passing on the small benchmark set. ML-assisted pass metrics were not collected in this run because ML flag was not enabled during this specific local evaluation call.

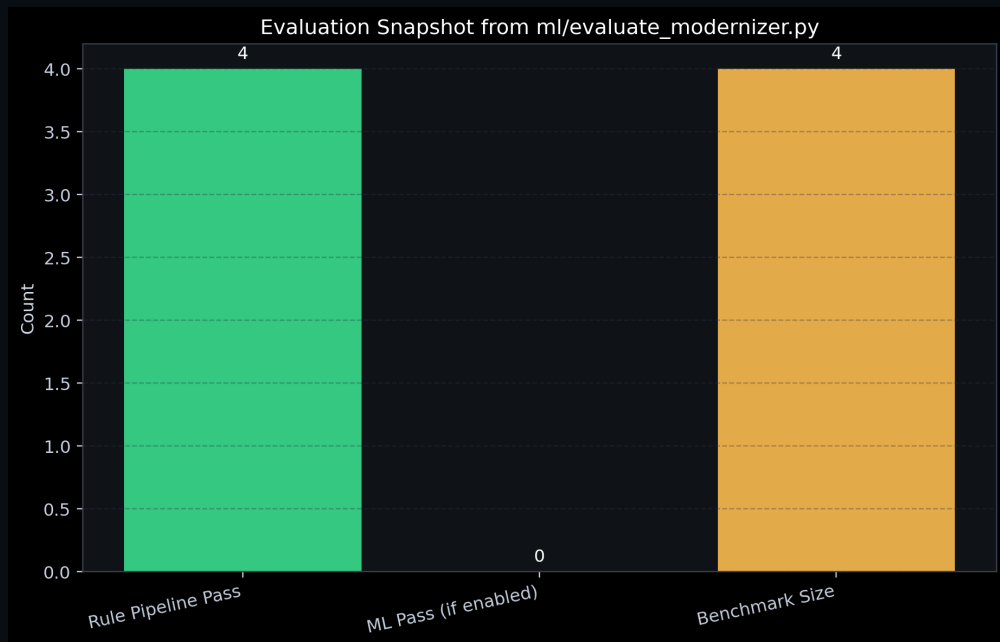


Figure 6. Evaluation summary from benchmark script.

Case	Rule OK	Validation	Expected modernization achieved
print_xrange	True	True	yes
raw_input	True	True	yes
except_syntax	True	True	yes
has_key	True	True	yes

12. Error Analysis and Failure Modes

Although benchmark pass is strong on covered patterns, realistic migrations can fail in several ways. The report includes failure modes explicitly to avoid overclaiming.

Observed and anticipated failure classes are listed below, along with mitigation status in current architecture.

Failure Class	Description	Current Mitigation	Residual Risk
Semantic drift	Modernized code changes behavior.	Validation: manual review warnings.	Medium
Unsupported archaic syntax	Creates constructs outside known patterns.	Pattern detection + conservative fallback.	Medium/High
Overfitting to seen templates	Model memorizes style and misses logic.	Model strategy + external data.	Medium
Unsafe dynamic execution	Eval/exec behavior can be risky.	Analyzer marks as blocked/restricted.	Low/Medium
Incomplete external data	Only local source currently active.	Manifest supports expansion.	Medium

13. Safety, Validation, and Production Guardrails

- All generated output is reparsed for syntax; syntax errors fail validation.
- Residual legacy constructs are explicitly scanned and blocked if present.
- Function-presence checks reduce accidental deletion of original APIs.
- Division/text-bytes warnings are surfaced for manual semantic review.
- Eval/exec patterns are assigned critical risk in analyzer mode logic.

This guardrail stack is the main reason Helix prefers hybrid modernization instead of unconstrained LLM rewrite.

In other words: model output can propose; deterministic validation disposes.

13. Reproducibility Runbook

This section provides a complete practical runbook to rebuild data, train model adapters, evaluate, and integrate with app runtime.

Commands are aligned with project scripts and tested paths.

```
# 1) Build seed and curated modernization data
python3 ml/build_seed_modernization_dataset.py
python3 ml/curate_dataset.py

# 2) Build weighted training mixture
python3 ml/build_training_mixture.py

# 3) Train (local or Colab wrapper style)
python3 ml/train_modernizer_unsloth.py --data-path ml/data/training_mixture.jsonl

# 4) Evaluate deterministic baseline
python3 ml/evaluate_modernizer.py

# 5) Evaluate with ML adapter (when runtime available)
python3 ml/evaluate_modernizer.py --enable-ml \
  --adapter-path ml/models/nebula-modernizer-qwen25-1.5b \
  --base-model Qwen/Qwen2.5-Coder-1.5B-Instruct
```

14. Colab Execution Notes and Constraints

- Preferred runtime: T4/L4 GPU in Google Colab.
- Use fp16 on T4; bf16 may fail on non-Ampere setups.
- Upload training_mixture.jsonl manually when notebook asks.
- Export adapter and place under ml/models/nebula-modernizer-qwen25-1.5b.
- Run benchmark after integrating adapter to validate no regression.

15. Integration with Agentic Pipeline

ML integration is intentionally optional and gated. The app can run with `ML\_MODEL\_ENABLED=false` and still modernize through deterministic rules. This is valuable for reliability-first demos and offline grading.

When enabled, ML reasoner is expected to support ambiguous transformations, but validation remains mandatory. This prevents silent quality regression from model drift.

Environment Variable	Purpose
ML_MODEL_ENABLED	Toggle model-assisted modernization path.
ML_MODEL_BASE	Specify base model identifier at runtime.
ML_MODEL_ADAPTER_PATH	Point app to local LoRA adapter folder.

16. Limitations and Next Iterations

- Current training mixture is single-source in active run due absent external prepared pairs.
- Benchmark set is compact (4 cases) and should be expanded to production-like coverage.
- Loss reduction is strong, but behavioral equivalence testing is still limited.
- No automatic runtime test harness exists yet for transformed code execution safety.
- Model card generated by trainer is generic and should be rewritten with task-specific details.

Recommended next milestone is evaluation expansion: build 100+ legacy snippets across syntax, API, and semantic-risk categories; report pass/fail with strict criteria and manual audit notes.

Second milestone is external-source activation in mixture builder and ablation runs (local-only vs mixed data).

17. Data Distribution Deep Dive

Beyond aggregate counts, model quality depends on data-shape properties. Input/output length spread, tag concentration, and source homogeneity directly affect convergence behavior and generalization risk.

In the current run, the mixture is dominated by one source (`'local_curated_modernization'`), which is transparent but increases distribution-shift risk for unseen legacy styles.

A robust next-step would add external prepared sources and run per-source ablation studies to quantify generalization delta.

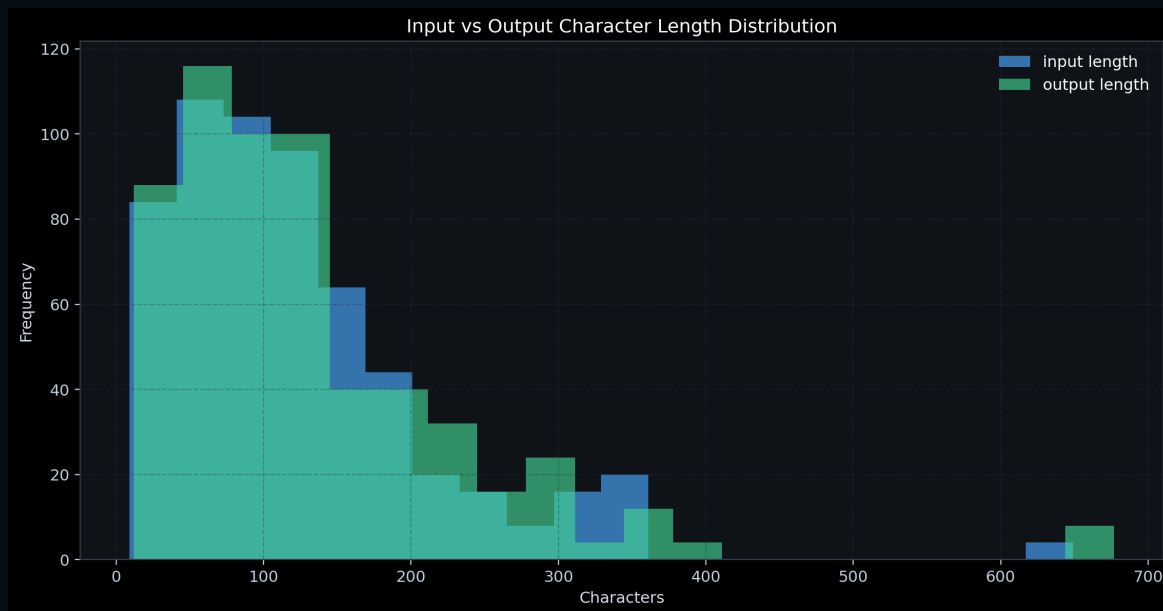


Figure 7. Histogram of input/output character lengths across training mixture.



Figure 8. Step-to-step loss delta used to inspect optimization stability.

Statistic	Value
Mixture row count	584
Input char mean / median / p90	123.6 / 104.0 / 252.0
Output char mean / median / p90	132.3 / 108.5 / 265.0
Source diversity count	1
Risk labels present	high, low, medium

18. Detailed Checkpoint Log Analysis

This section presents per-step checkpoint telemetry from ``checkpoint-292/trainer_state.json``. It is intentionally verbose to support viva-level technical defense and troubleshooting.

Each row records step, epoch fraction, loss, learning rate, and gradient norm. Reviewing this table helps diagnose instability, scheduler errors, and gradient spikes.

Checkpoint-292 Log Segment 1/4

Step	Epoch	Loss	Learning Rate	Grad Norm
5	0.0342	2.235983	0.000053333	1.197382
10	0.0685	1.711869	0.000120000	0.940137
15	0.1027	1.022831	0.000186667	0.973224
20	0.1370	0.586555	0.000199897	0.720146
25	0.1712	0.463857	0.000199480	0.569514
30	0.2055	0.447989	0.000198742	0.487895
35	0.2397	0.417337	0.000197687	0.355484
40	0.2740	0.536770	0.000196318	0.701058
45	0.3082	0.383374	0.000194640	0.418568
50	0.3425	0.387937	0.000192657	0.298585
55	0.3767	0.366870	0.000190376	0.445869
60	0.4110	0.339563	0.000187805	0.326957
65	0.4452	0.353375	0.000184951	0.416493
70	0.4795	0.351094	0.000181825	0.624199
75	0.5137	0.314827	0.000178435	0.283837
80	0.5479	0.331319	0.000174793	0.427379
85	0.5822	0.363923	0.000170911	0.437947
90	0.6164	0.343279	0.000166801	0.298927

Checkpoint-292 Log Segment 2/4

Step	Epoch	Loss	Learning Rate	Grad Norm
95	0.6507	0.272429	0.000162476	0.412388
100	0.6849	0.311279	0.000157950	0.375976
105	0.7192	0.345261	0.000153238	0.457279
110	0.7534	0.337304	0.000148354	0.468304
115	0.7877	0.269259	0.000143315	0.464663
120	0.8219	0.276559	0.000138137	0.488282
125	0.8562	0.235108	0.000132837	0.317076
130	0.8904	0.213830	0.000127430	0.394184
135	0.9247	0.239013	0.000121936	0.484446
140	0.9589	0.230641	0.000116371	0.925487
145	0.9932	0.226046	0.000110754	0.635669
150	1.0274	0.171890	0.000105101	0.604753
155	1.0616	0.201497	0.000099433	0.474070
160	1.0959	0.144978	0.000093766	0.483131
165	1.1301	0.141568	0.000088120	0.601276
170	1.1644	0.131392	0.000082511	0.576219
175	1.1986	0.141935	0.000076959	0.409619
180	1.2329	0.132694	0.000071481	0.759875

Checkpoint-292 Log Segment 3/4

Step	Epoch	Loss	Learning Rate	Grad Norm
185	1.2671	0.108933	0.000066094	0.340166
190	1.3014	0.138582	0.000060817	0.713576
195	1.3356	0.111237	0.000055665	0.497048
200	1.3699	0.117508	0.000050656	0.842663
205	1.4041	0.125399	0.000045806	0.565256
210	1.4384	0.102850	0.000041130	0.509805
215	1.4726	0.159554	0.000036643	1.358052
220	1.5068	0.088294	0.000032360	0.400698
225	1.5411	0.128638	0.000028294	0.701752
230	1.5753	0.105060	0.000024459	0.545043
235	1.6096	0.104683	0.000020866	0.481660
240	1.6438	0.115703	0.000017529	0.613491
245	1.6781	0.085086	0.000014456	0.463221
250	1.7123	0.081800	0.000011658	0.478428
255	1.7466	0.093564	0.000009144	0.581622
260	1.7808	0.095885	0.000006922	0.406455
265	1.8151	0.099103	0.000005000	0.540141
270	1.8493	0.080657	0.000003383	0.598865

Checkpoint-292 Log Segment 4/4

Step	Epoch	Loss	Learning Rate	Grad Norm
275	1.8836	0.081996	0.000002077	0.485535
280	1.9178	0.100720	0.000001085	0.588686
285	1.9521	0.079997	0.000000411	0.409372
290	1.9863	0.085461	0.000000058	0.789504

19. Pattern-Level Coverage Matrix

This matrix links three perspectives: (1) deterministic rule support in pipeline, (2) observed training tag frequency, and (3) benchmark presence. This helps identify coverage gaps systematically.

Pattern	Deterministic Rule	Train Tag Count	Benchmark Presence	Notes
print_statement	yes	36	yes	core syntax rewrite
xrange	yes	4	yes	iterator API migration
raw_input	yes	4	yes	input API migration
except_as	yes	4	yes	exception binding syntax
has_key	yes	4	yes	dictionary membership migration
iteritems	yes	4	no	dict iterator API
iterkeys	yes	4	no	dict iterator API
itervalues	yes	4	no	dict iterator API
unicode	yes	4	no	text type migration
long	yes	4	no	integer type migration
basestring	yes	4	no	type alias migration
backtick_repr	yes	0	no	Python 1.x/2.x repr shorthand
apply_builtin	yes	0	no	legacy function-apply idiom

20. Sample-by-Sample Modernization Casebook

This casebook section provides concrete before/after examples from the active datasets. It is included to make training intent auditable and to support examiner-level discussion of what the model is expected to learn.

Case 1: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: low

```
Input:
print "hello world"
Target Output:
print("hello world")
```

Case 2: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: low

```
Input:
for i in xrange(5):
    print i
Target Output:
for i in range(5):
    print(i)
```

Case 3: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: low

```
Input:
name = raw_input("Name: ")
print name
Target Output:
name = input("Name: ")
print(name)
```

Case 4: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: low

```
Input:
try:
    risky()
except ValueError, exc:
    print exc
Target Output:
try:
    risky()
except ValueError as exc:
    print(exc)
```

Case 5: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: low

```
Input:
if a <> b:
    print a
Target Output:
if a != b:
    print(a)
```

Case 6: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: low

```
Input:
for key, value in data.iteritems():
    print key, value
Target Output:
for key, value in data.items():
    print(key, value)
```

Case 7: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: medium

```
Input:
if config.has_key('debug'):
    print config['debug']
Target Output:
if 'debug' in config:
    print(config['debug'])
```

Case 8: Modernize this legacy Python code to current Python while preserving behavior.

Source: local | Risk label: medium

```
Input:
value = long(10)
print unicode(value)
Target Output:
value = int(10)
print(str(value))
```

Case 9: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
print 'Hello World'
for i in range(10):
    print i

Target Output:
print('Hello World')
for i in range(10):
    print(i)
```

Case 10: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
import thread
def my_thread_function():
    print 'Hello, world!'
thread.start_new_thread(my_thread_function, ())
for i in range(10):
    print i
thread.exit_thread()

Target Output:
import threading
def my_thread_function():
    print('Hello, world!')
threading.Thread(target=my_thread_function).start()
for i in range(10):
    print(i)
```

Case 11: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
def calculate_area_old(base, height):
    return base * height
area = calculate_area_old(10, 20)
print area

Target Output:
def calculate_area(base: float, height: float) -> float:
    return base * height
print(calculate_area(10, 20))
```

Case 12: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
import types

# Define a class with a method
class MyClass:
    def __init__(self):
        self.myvar = types.IntType(5)

# Create an instance of the class
obj = MyClass()

# Print the value of myvar
print obj.myvar
Target Output:
import typing

class MyClass:
    def __init__(self) -> None:
        self.myvar: int = 5

# Create an instance of the class
obj = MyClass()

# Print the value of myvar
print(obj.myvar)
```

Case 13: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
for i in range(10):
    print i
print "Finished"
Target Output:
for i in range(10):
    print(i)
print("Finished")
```

Case 14: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
def add(x, y):
    result = 0
    while result < 10000:
        result += x
        result += y
    return result
print add(5,10)
Target Output:
def add(x: int, y: int) -> int:
    result = 0
    while result < 10000:
        result += x
        result += y
    return result
print(add(5, 10))
```

Case 15: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
try:
    with open('file.txt', 'r') as f:
        data = f.read()
except IOError:
    print 'Error reading file'
Target Output:
try:
    with open('file.txt', 'r') as f:
        data = f.read()
except FileNotFoundError:
    print('Error reading file')
```

Case 16: Modernize this legacy Python code to current Python while preserving behavior.

Source: phase2_refactoring_dataset | Risk label: low

```
Input:
def sort_list(input_list):
    input_list.sort()
    return input_list
input_list = [3, 6, 8, 10, 1, 2, 1]
result = sort_list(input_list)
print result
Target Output:
def sort_list(input_list):
    return sorted(input_list)
input_list = [3, 6, 8, 10, 1, 2, 1]
result = sort_list(input_list)
print(result)
```

Appendix A: Full Command Reference

```
python3 ml/build_seed_modernization_dataset.py
python3 ml/curate_dataset.py
python3 ml/build_training_mixture.py
python3 ml/train_modernizer_unsloth.py --data-path ml/data/training_mixture.jsonl
python3 ml/train_modernizer_colab.py --data-path /content/training_mixture.jsonl
python3 ml/evaluate_modernizer.py
python3 ml/evaluate_modernizer.py --enable-ml --adapter-path ml/models/nebula-modernizer-qwen25-1.5b --base-model Qwen/Qwen2
python3 ml/infer_modernizer.py --adapter-path ml/models/nebula-modernizer-qwen25-1.5b --code "for i in xrange(3):\n    print
python3 -m uvicorn main:app --reload
```

Appendix B: Key Code Excerpts

train_modernizer_unsloth.py (core setup)

```
0052:
0053: def main() -> None:
0054:     try:
0055:         from datasets import Dataset
0056:         from trl import SFTConfig, SFTTrainer
0057:         from unsloth import FastLanguageModel
0058:     except ImportError as exc:
0059:         raise SystemExit(
0060:             "Missing ML dependencies. Install with `pip install -r requirements-ml.txt`."
0061:         ) from exc
0062:
0063:     parser = argparse.ArgumentParser()
0064:     parser.add_argument("--base-model", default=DEFAULT_BASE_MODEL)
0065:     parser.add_argument("--data-path", default=str(DEFAULT_DATA_PATH))
0066:     parser.add_argument("--output-dir", default=str(DEFAULT_OUTPUT_DIR))
0067:     parser.add_argument("--batch-size", type=int, default=2)
0068:     parser.add_argument("--gradient-accumulation-steps", type=int, default=4)
0069:     parser.add_argument("--epochs", type=int, default=3)
0070:     parser.add_argument("--learning-rate", type=float, default=2e-4)
0071:     args = parser.parse_args()
0072:
0073:     data_path = Path(args.data_path)
0074:     output_dir = Path(args.output_dir)
0075:
0076:     if not data_path.exists():
0077:         raise SystemExit(
0078:             f"Training data not found at {data_path}. Run `python3 ml/build_seed_modernization_dataset.py` "
0079:             f"and `python3 ml/curate_dataset.py` first."
0080:         )
0081:
0082:     global DATA_PATH
0083:     DATA_PATH = data_path
0084:     raw_records = load_records()
0085:     dataset = Dataset.from_list(raw_records)
0086:     dataset = dataset.shuffle(seed=42)
0087:     dataset = dataset.map(formatting_prompts_func, batched=True)
0088:
0089:     model, tokenizer = FastLanguageModel.from_pretrained(
0090:         model_name=args.base_model,
0091:         max_seq_length=MAX_SEQ_LENGTH,
0092:         dtype=None,
0093:         load_in_4bit=True,
0094:     )
0095:
0096:     model = FastLanguageModel.get_peft_model(
0097:         model,
0098:         r=16,
0099:         target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
0100:         lora_alpha=16,
0101:         lora_dropout=0,
0102:         bias="none",
0103:         use_gradient_checkpointing="unsloth",
0104:         random_state=42,
0105:     )
0106:
0107:     trainer = SFTTrainer(
0108:         model=model,
0109:         tokenizer=tokenizer,
0110:         train_dataset=dataset,
0111:         dataset_text_field="text",
0112:         max_seq_length=MAX_SEQ_LENGTH,
0113:         args=SFTConfig(
0114:             output_dir=str(output_dir),
```

```

0115:         per_device_train_batch_size=args.batch_size,
0116:         gradient_accumulation_steps=args.gradient_accumulation_steps,
0117:         learning_rate=args.learning_rate,
0118:         num_train_epochs=args.epochs,
0119:         warmup_ratio=0.05,
0120:         logging_steps=5,
0121:         save_strategy="epoch",
0122:         lr_scheduler_type="cosine",
0123:         weight_decay=0.01,
0124:         report_to=[],
0125:     ),
0126: )
0127:
0128: trainer.train()
0129: model.save_pretrained(str(output_dir))
0130: tokenizer.save_pretrained(str(output_dir))
0131: print(f"Saved adapter to {output_dir}")
0132:
0133:
0134: if __name__ == "__main__":
0135:     main()

```

build_training_mixture.py (weighting logic)

```

0001: import json
0002: from pathlib import Path
0003:
0004:
0005: MANIFEST_PATH = Path("ml/public_dataset_manifest.json")
0006: OUTPUT_PATH = Path("ml/data/training_mixture.jsonl")
0007:
0008:
0009: def load_jsonl(path: Path) -> list[dict]:
0010:     records = []
0011:     if not path.exists():
0012:         return records
0013:     with path.open(encoding="utf-8") as handle:
0014:         for line in handle:
0015:             line = line.strip()
0016:             if line:
0017:                 records.append(json.loads(line))
0018:     return records
0019:
0020:
0021: def main() -> None:
0022:     manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
0023:     OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)
0024:
0025:     mixture = []
0026:     for source in manifest["sources"]:
0027:         path_value = source.get("path") or source.get("prepared_path")
0028:         if not path_value:
0029:             continue
0030:
0031:         path = Path(path_value)
0032:         if not path.exists():
0033:             print(f"Skipping missing source: {path}")
0034:             continue
0035:
0036:         records = load_jsonl(path)
0037:         weight = int(source.get("weight", 1))
0038:         for record in records:
0039:             enriched = dict(record)
0040:             enriched["mixture_source"] = source["name"]
0041:             for _ in range(max(weight, 1)):
0042:                 mixture.append(enriched)
0043:
0044:         with OUTPUT_PATH.open("w", encoding="utf-8") as handle:
0045:             for record in mixture:
0046:                 handle.write(json.dumps(record, ensure_ascii=True) + "\n")
0047:
0048:         print(f"Wrote {len(mixture)} training examples to {OUTPUT_PATH}")
0049:
0050:
0051: if __name__ == "__main__":
0052:     main()

```

evaluate_modernizer.py (benchmark flow)

```

0001: import argparse

```

```

0002: import asyncio
0003: import json
0004: from pathlib import Path
0005: import sys
0006:
0007: ROOT = Path(__file__).resolve().parent.parent
0008: if str(ROOT) not in sys.path:
0009:     sys.path.insert(0, str(ROOT))
0010:
0011: from agents.analyzer import AnalyzerAgent
0012: from agents.critic import CriticAgent
0013: from agents.planner import PlannerAgent
0014: from agents.suggester import SuggestionAgent
0015: from core.execution import ExecutionCore
0016: from core.ml_reasoner import MLReasoner
0017: from core.validation import ValidationCore
0018:
0019:
0020: DEFAULT_BENCHMARK = Path("ml/data/eval_benchmark.jsonl")
0021:
0022:
0023: def load_jsonl(path: Path) -> list[dict]:
0024:     rows = []
0025:     with path.open(encoding="utf-8") as handle:
0026:         for line in handle:
0027:             line = line.strip()
0028:             if line:
0029:                 rows.append(json.loads(line))
0030:     return rows
0031:
0032:
0033: async def run_rule_pipeline(code: str) -> dict:
0034:     analysis = await AnalyzerAgent().process(code)
0035:     suggestions = await SuggestionAgent().process(analysis, code)
0036:     critiques = await CriticAgent().process(suggestions)
0037:     plan = await PlannerAgent().process(suggestions, critiques)
0038:     output = ExecutionCore().apply_changes(code, plan.get("selected_plans", []))
0039:     validation = ValidationCore().validate(code, output)
0040:     return {"analysis": analysis, "plan": plan, "output": output, "validation": validation}
0041:
0042:
0043: def contains_all(text: str, expected: list[str]) -> bool:
0044:     return all(item in text for item in expected)
0045:
0046:
0047: def main() -> None:
0048:     parser = argparse.ArgumentParser()
0049:     parser.add_argument("--benchmark", default=str(DEFAULT_BENCHMARK))
0050:     parser.add_argument("--enable-ml", action="store_true")
0051:     parser.add_argument("--adapter-path", default="ml/models/nebula-modernizer-qwen25-1.5b")
0052:     parser.add_argument("--base-model", default="Qwen/Qwen2.5-Coder-7B-Instruct")
0053:     args = parser.parse_args()
0054:
0055:     benchmark = load_jsonl(Path(args.benchmark))
0056:     reasoner = MLReasoner(args.enable_ml, args.adapter_path, args.base_model)
0057:
0058:     rule_pass = 0
0059:     ml_pass = 0
0060:     rows = []
0061:     for item in benchmark:
0062:         rule_result = asyncio.run(run_rule_pipeline(item["input"]))
0063:         rule_ok = rule_result["validation"]["success"] and contains_all(rule_result["output"], item["expected_contains"])
0064:         if rule_ok:
0065:             rule_pass += 1
0066:
0067:         ml_ok = False
0068:         ml_output = ""
0069:         if args.enable_ml:
0070:             ml_result = reasoner.modernize(item["input"])
0071:             ml_output = ml_result.output
0072:             ml_ok = ml_result.available and contains_all(ml_output, item["expected_contains"])
0073:             if ml_ok:
0074:                 ml_pass += 1
0075:
0076:         rows.append(
0077:             {
0078:                 "name": item["name"],
0079:                 "rule_ok": rule_ok,
0080:                 "rule_validation": rule_result["validation"]["success"],
0081:                 "ml_ok": ml_ok,
0082:                 "rule_output": rule_result["output"],
0083:                 "ml_output": ml_output,
0084:             }
0085:         )

```

```
0086:
0087:     print(json.dumps(
0088:         {
0089:             "benchmark_size": len(benchmark),
0090:             "rule_pass": rule_pass,
0091:             "ml_pass": ml_pass if args.enable_ml else None,
0092:             "details": rows,
0093:         },
0094:         indent=2,
0095:     ))
0096:
0097:
0098: if __name__ == "__main__":
0099:     main()
```

Appendix C: Dataset Snapshots

Below are abbreviated examples from active dataset files for auditability. Long payloads are trimmed to keep report readable.

File	Instruction (trimmed)	Input snippet (trimmed)	Output snippet (trimmed)
seed_modernization_dataset.jsonl	Modernize this legacy	Python's old dev world, I've Python with the less solid behavior.	
modernization_train.jsonl	Modernize this legacy	Python's old dev world, I've Python with the less solid behavior	range(10):\n print(i)
training_mixture.jsonl	Modernize this legacy	Python's old dev world, I've Python with the less solid behavior	range(10):\n print(i)

Appendix D: Full Training Log Dump

Raw training telemetry (checkpoint-292) is presented below in tabular segments for full transparency.

Log Dump Segment 1/3

Step	Epoch	Loss	Learning Rate	Grad Norm
5	0.0342	2.235983	0.000053333	1.197382
10	0.0685	1.711869	0.000120000	0.940137
15	0.1027	1.022831	0.000186667	0.973224
20	0.1370	0.586555	0.000199897	0.720146
25	0.1712	0.463857	0.000199480	0.569514
30	0.2055	0.447989	0.000198742	0.487895
35	0.2397	0.417337	0.000197687	0.355484
40	0.2740	0.536770	0.000196318	0.701058
45	0.3082	0.383374	0.000194640	0.418568
50	0.3425	0.387937	0.000192657	0.298585
55	0.3767	0.366870	0.000190376	0.445869
60	0.4110	0.339563	0.000187805	0.326957
65	0.4452	0.353375	0.000184951	0.416493
70	0.4795	0.351094	0.000181825	0.624199
75	0.5137	0.314827	0.000178435	0.283837
80	0.5479	0.331319	0.000174793	0.427379
85	0.5822	0.363923	0.000170911	0.437947
90	0.6164	0.343279	0.000166801	0.298927
95	0.6507	0.272429	0.000162476	0.412388
100	0.6849	0.311279	0.000157950	0.375976
105	0.7192	0.345261	0.000153238	0.457279
110	0.7534	0.337304	0.000148354	0.468304
115	0.7877	0.269259	0.000143315	0.464663
120	0.8219	0.276559	0.000138137	0.488282

Log Dump Segment 2/3

Step	Epoch	Loss	Learning Rate	Grad Norm
125	0.8562	0.235108	0.000132837	0.317076
130	0.8904	0.213830	0.000127430	0.394184
135	0.9247	0.239013	0.000121936	0.484446
140	0.9589	0.230641	0.000116371	0.925487
145	0.9932	0.226046	0.000110754	0.635669
150	1.0274	0.171890	0.000105101	0.604753
155	1.0616	0.201497	0.000099433	0.474070
160	1.0959	0.144978	0.000093766	0.483131
165	1.1301	0.141568	0.000088120	0.601276
170	1.1644	0.131392	0.000082511	0.576219
175	1.1986	0.141935	0.000076959	0.409619
180	1.2329	0.132694	0.000071481	0.759875
185	1.2671	0.108933	0.000066094	0.340166
190	1.3014	0.138582	0.000060817	0.713576
195	1.3356	0.111237	0.000055665	0.497048
200	1.3699	0.117508	0.000050656	0.842663
205	1.4041	0.125399	0.000045806	0.565256
210	1.4384	0.102850	0.000041130	0.509805
215	1.4726	0.159554	0.000036643	1.358052
220	1.5068	0.088294	0.000032360	0.400698
225	1.5411	0.128638	0.000028294	0.701752
230	1.5753	0.105060	0.000024459	0.545043
235	1.6096	0.104683	0.000020866	0.481660
240	1.6438	0.115703	0.000017529	0.613491

Log Dump Segment 3/3

Step	Epoch	Loss	Learning Rate	Grad Norm
245	1.6781	0.085086	0.000014456	0.463221
250	1.7123	0.081800	0.000011658	0.478428
255	1.7466	0.093564	0.000009144	0.581622
260	1.7808	0.095885	0.000006922	0.406455
265	1.8151	0.099103	0.000005000	0.540141
270	1.8493	0.080657	0.000003383	0.598865
275	1.8836	0.081996	0.000002077	0.485535
280	1.9178	0.100720	0.000001085	0.588686
285	1.9521	0.079997	0.000000411	0.409372
290	1.9863	0.085461	0.000000058	0.789504

Appendix E: Colab Cell-by-Cell Guide

This guide maps the practical notebook flow used for LoRA fine-tuning on Colab T4/L4 GPU.

Each block is intentionally short so viva examiners can follow the complete lifecycle quickly.

Cell 1 - Runtime check
`import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'No GPU')`

Cell 2 - Install dependencies
`!pip -q install --upgrade unsloth datasets peft trl accelerate sentencepiece transformers`

Cell 3 - Upload dataset
`from google.colab import files
uploaded = files.upload()
DATA_PATH = '/content/training_mixture.jsonl'`

Cell 4 - Build Dataset object and prompt formatting
`load jsonl -> Dataset.from_list(...) -> shuffle -> map(formatting_prompts_func)`

Cell 5 - Load base model in 4-bit and attach LoRA
`FastLanguageModel.from_pretrained(... load_in_4bit=True)
FastLanguageModel.get_peft_model(r=16, target_modules=[...])`

Cell 6 - Trainer config
`SFTConfig(output_dir=..., batch_size, grad_accum, lr, epochs, scheduler='cosine')`

Cell 7 - Train and save
`trainer.train()
model.save_pretrained(output_dir); tokenizer.save_pretrained(output_dir)`

Cell 8 - Package and download adapter
`zip adapter directory and download to local machine`

Cell 9 - Local integration
`Place adapter under ml/models/nebula-modernizer-qwen25-1.5b
Run ml/evaluate_modernizer.py and app endpoints.`

Appendix F: File Inventory (ML Folder)

Path	Type	Approx Size
COLAB_TRAINING.md	file	1.6 KB
Helix_AI_Modernizer_Colab.ipynb	file	6.6 KB
README.md	file	4.7 KB
build_seed_modernization_dataset.py	file	4.7 KB
build_training_mixture.py	file	1.4 KB
curate_dataset.py	file	5.2 KB
data	directory	
evaluate_modernizer.py	file	3.1 KB
external	directory	
infer_modernizer.py	file	1.8 KB
models	directory	
package_adapter.py	file	0.6 KB
prepare_bugsinpy_pairs.py	file	1.7 KB
prepare_gumtree_python_pairs.py	file	1.6 KB
prepare_quixbugs_pairs.py	file	1.4 KB
prepare_swebench_lite.py	file	1.6 KB
public_dataset_manifest.json	file	1.4 KB
train_modernizer_colab.py	file	1.4 KB
train_modernizer_unsloth.py	file	3.9 KB

End of report.

This document can be regenerated with: ``python3 docs/generate_ml_training_in_depth_report.py``